Reinforcement learning

Robot học chính sách từ phần thưởng

Bài toán robot chuyển sách

Robot phải tới bàn trả sách qua một lưới có một ô chắn.

Mỗi lần chỉ được thử một hành động, rồi quan sát vị trí mới và phần thưởng.

Một bước xa đích biết nó đang giúp ích bằng cách nào?

CẦU NỐI TỪ CP

Biết bản đồ và chỉ cần đường ngắn nhất: dùng BFS.

Chưa biết kết quả hành động: thu trải nghiệm, rồi cập nhật cách chọn.

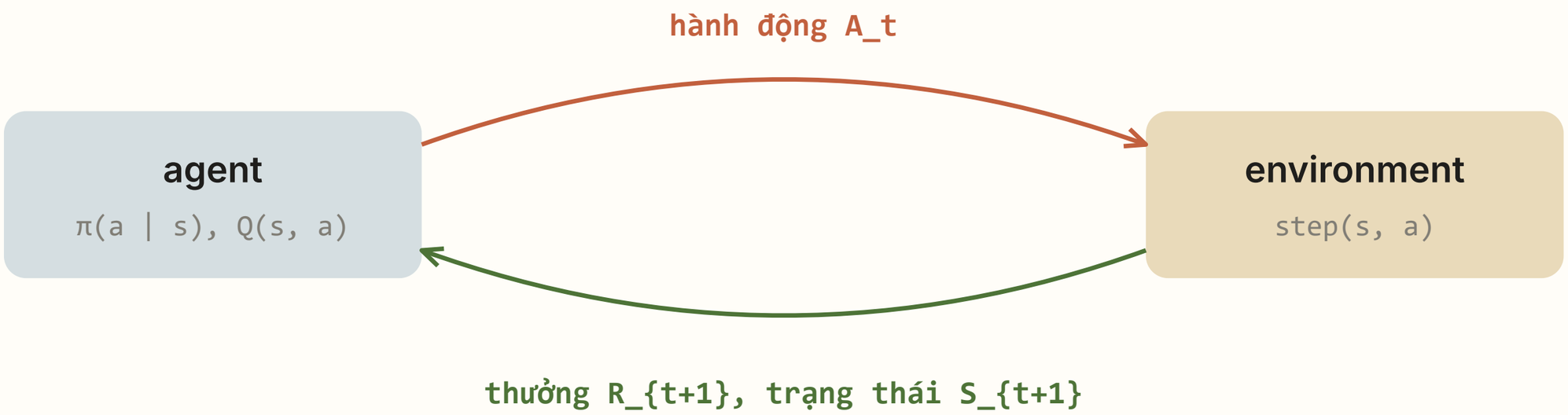
Mục tiêu buổi này

- Mô tả một MDP nhỏ và kiểm điều kiện kết thúc.
- Tính return và một bước Bellman.
- Cài Q-learning rồi đọc một đường đi thật.
- Hiểu policy gradient trên một nhiệm vụ một bước.

01

Tương tác và mục tiêu

Vòng tương tác của một MDP



Vòng tương tác: agent chọn hành động theo trạng thái hiện tại; environment trả phần thưởng và trạng thái mới. Agent học khi trải nghiệm làm thay đổi Q hoặc policy.

Trạng thái phải đủ cho bước tiếp theo

: trạng thái trước hành động.

: hành động.

, : kết quả sau hành động.

Tính Markov: khi biết trạng thái và hành động hiện tại, phân phối của trạng thái và phần thưởng kế tiếp không cần lịch sử trước đó.

Pin ảnh hưởng di chuyển: thêm mức pin vào trạng thái.

Thời hạn thuộc nhiệm vụ: biểu diễn thời gian còn lại.

HỎI NHANH

Robot chỉ ghi lại vị trí ô. Khi pin yếu, lệnh “tiến” có lúc không làm robot di chuyển. Nên sửa mô tả trạng thái thế nào?

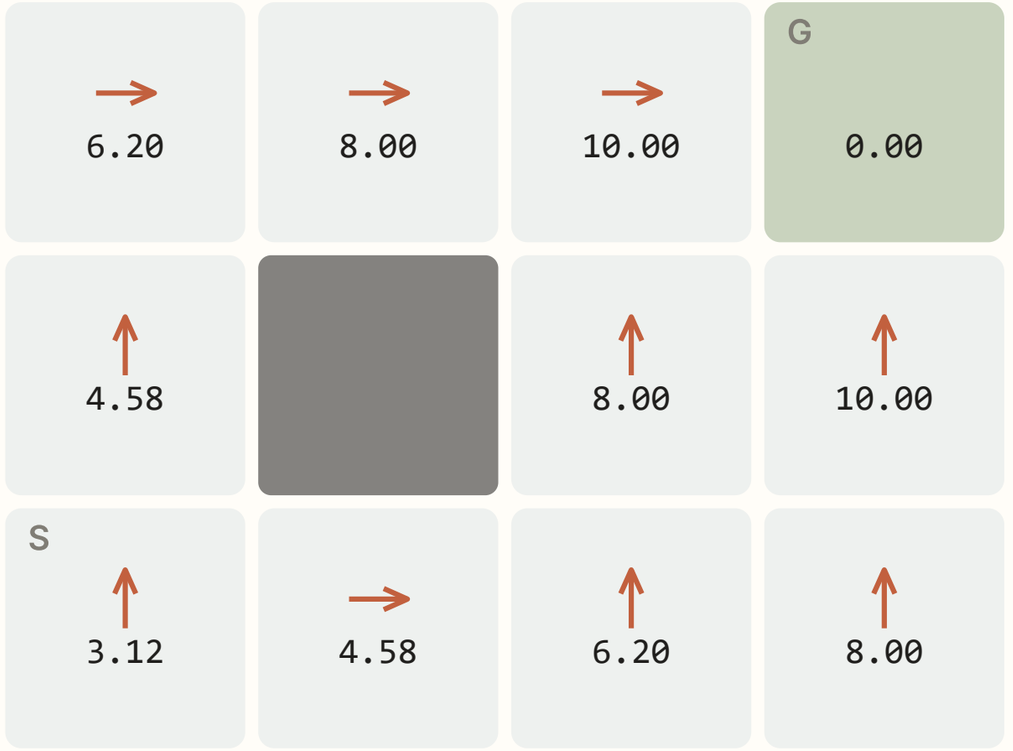
A Đổi seed mỗi episode để trung bình hóa ảnh hưởng của pin

B Thêm mức pin hiện tại vào trạng thái

C Đưa toàn bộ chuỗi ô đã đi qua vào trạng thái

D Giữ nguyên vị trí, chỉ tăng số episode để có đủ mẫu

Gridworld dùng để kiểm bằng tay



mỗi bước −1

vào G +10, kết thúc

γ = 0.9

V*(S) = 3.122

Grid world của bài với V* tính bằng value iteration (từ code) và một hành động tối ưu ở mỗi ô; ô đen bị chắn. Từ S cần 5 bước, V*(S) = 3.122.

HỎI NHANH

Đổi luật chơi: robot có hướng nhìn là N, E, S hoặc W, và chỉ còn ba hành động là quay trái, quay phải, và tiến một ô theo hướng đang nhìn. Trên lưới của bài, MDP mới có bao nhiêu trạng thái và bao nhiêu hành động?

A 11 trạng thái, 3 hành động

B 48 trạng thái, 3 hành động

C 44 trạng thái, 4 hành động

D 44 trạng thái, 3 hành động

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q3.1

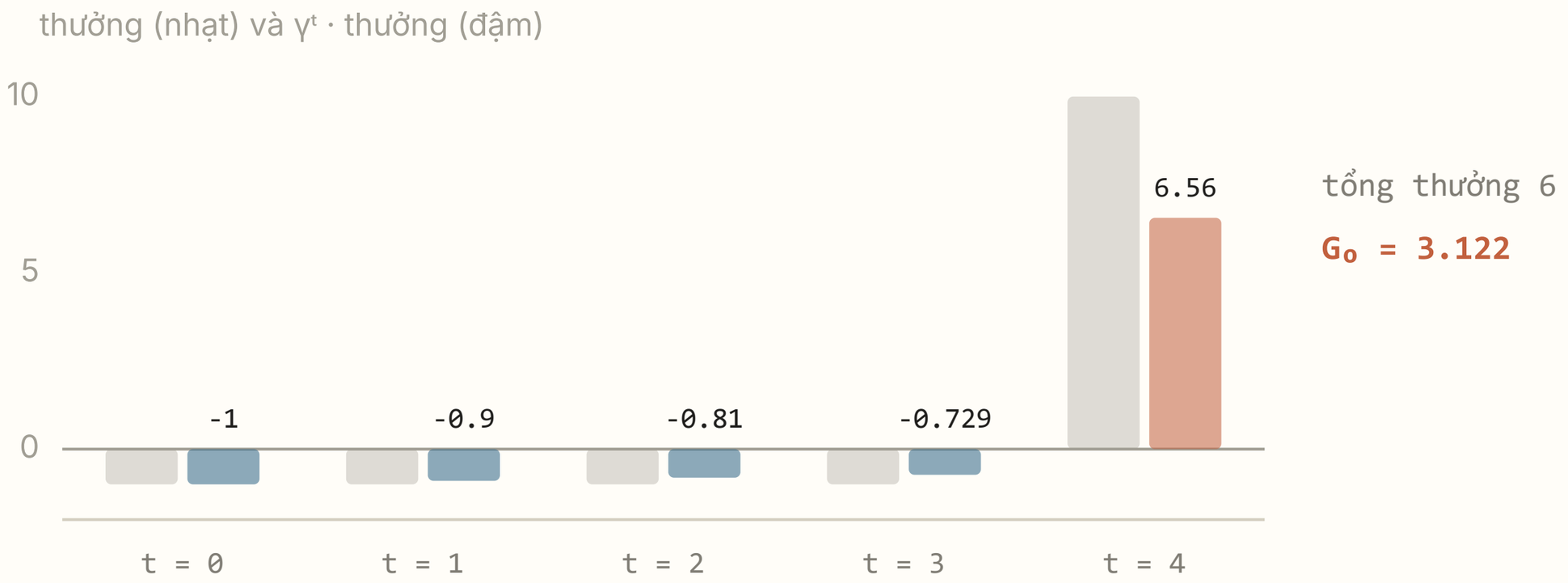
Return tính cả phần thưởng tương lai

Đường 5 bước: .

Với , phần thưởng càng xa càng có trọng số nhỏ.

Tổng không chiết khấu là 6 điểm. không phải learning rate.

Chiết khấu từng phần thưởng



Đường 5 bước của bài: bốn thưởng −1 rồi +10. Nhân mỗi thưởng với 0.9^t, thưởng 10 đến sau bốn lần chiết khấu chỉ còn 6.56, và return $G_0 = 3.122$ trong khi tổng thưởng là 6.

HỎI NHANH

Một bước đi vào terminal nhận thưởng 10. Return tính từ ngay trước bước đó bằng bao nhiêu?

A , vì thưởng của bước này bị nhân với

B , vì terminal không còn tương lai

C , vì thưởng đầu tiên có hệ số

D , vì còn của bước sau

Thưởng và định nghĩa “tối ưu”

Mỗi bước : đường dài mất thêm điểm.

: thưởng 10 đến muộn có giá trị nhỏ hơn.

Cả hai đẩy robot đi đường ngắn. Thưởng dương cho mỗi bước có thể khiến robot muốn kéo dài episode.

HỎI NHANH

Đổi thiết kế: vào đích nhận 10, mọi bước khác nhận 0, và . Policy nào là tối ưu?

A Chỉ các đường 5 bước, vì chúng tới đích sớm nhất

B Mọi policy chắc chắn tới đích, dài hay ngắn đều như nhau

C Policy đứng yên mãi, vì không bước nào bị phạt điểm

D Đường dài nhất, vì đi lâu hơn mà vẫn không mất điểm

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q3.8

02

Bellman và planning

Bellman nối việc xa với việc gần

vì sau terminal không còn phần thưởng tương lai.

Ở môi trường ngẫu nhiên, lấy kỳ vọng kết quả của từng hành động, rồi chọn `max`.

của một policy cố định: bỏ `max`, lấy trung bình theo policy. Có vòng thì đó là một hệ phương trình tuyến tính.

Value iteration tính ngược từ đích

Khoảng cách tới G	Giá trị tối ưu
1 bước	10
2 bước	8
3 bước	6.2
4 bước	4.58
5 bước	3.122

Mỗi bước lùi thêm: , với .

```
print(f"Value iteration tại S: {V[start]:.3f}")
print(f"Giá trị terminal: {V[goal]:.3f}")
```

KẾT QUẢ

```
Value iteration tại S: 3.122
Giá trị terminal: 0.000
```

Value iteration xét mọi hành động bằng mô hình đã biết. Đây là **planning**, chưa phải học từ mẫu.

Value iteration: chi phí và tốc độ lan

Một vòng xét mọi , mọi , mọi :

- bảng đầy đủ: ;
- lưới tất định: .

Đồ thị có vòng nên không có thứ tự tính một lần là xong.

Bắt đầu từ , mỗi vòng đưa thông tin từ đích lùi thêm một bước.

HỎI NHANH

Chạy value iteration đồng bộ trên lưới của bài, bắt đầu từ . Sau đúng 2 vòng, giá trị tại S bằng bao nhiêu?

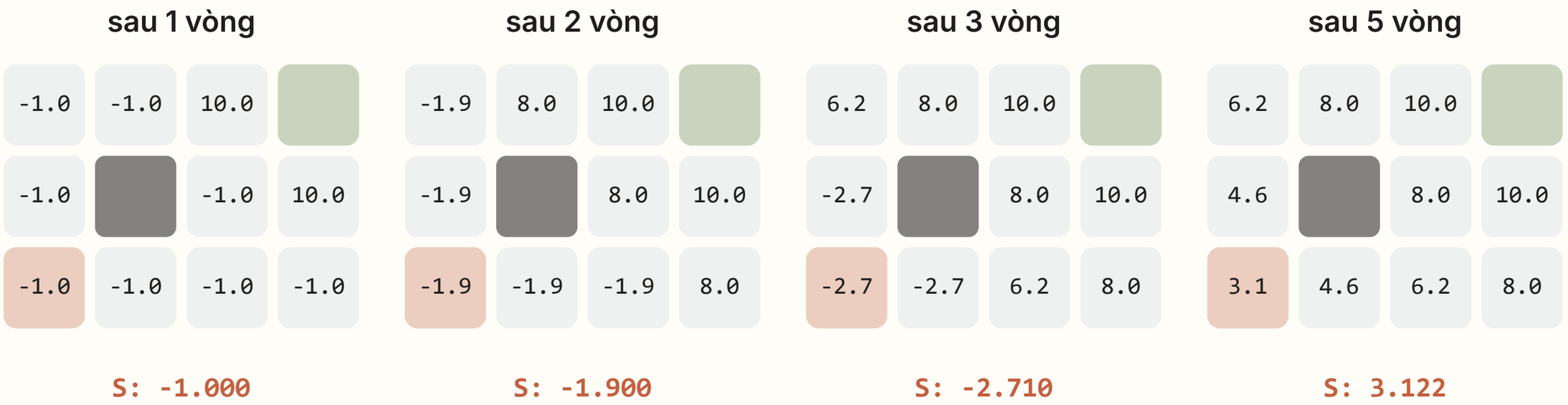
A

B

C

D

Thông tin lan từ đích



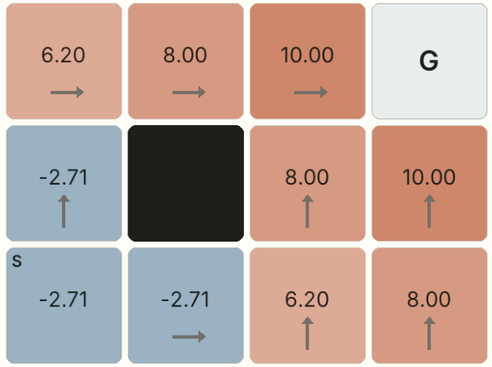
mỗi vòng đưa thông tin từ G lùi thêm một bước

Value iteration đồng bộ của bài từ V = 0 (tính lại bằng step của bài): sau 1 vòng S là −1, sau 2 vòng −1.9, và chỉ từ vòng 5 thưởng ở G mới lan tới S, cho 3.122.

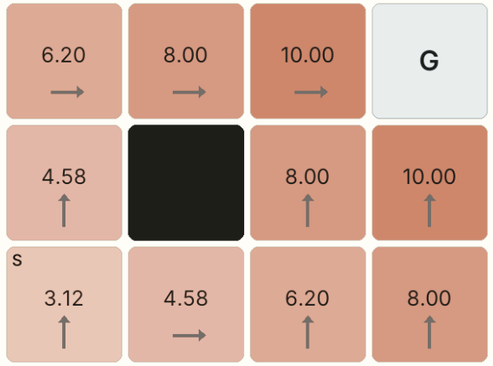
Giá trị lan từ đích về S



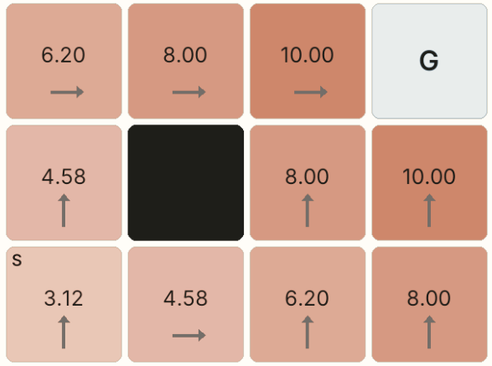
vòng 1: V(S) = -1.000



vòng 3: V(S) = -2.710



vòng 5: V(S) = 3.122



hội tụ: V(S) = 3.122, mũi tên là hành động tốt nhất ở mỗi ô

Value iteration của bài: mỗi vòng, mỗi ô lấy thưởng tốt nhất của một bước cộng giá trị chiết khấu của ô kế; giá trị lan dần từ đích G về S.

03

Học từ trải nghiệm

Q-learning cập nhật từng cặp

Bảng `(11, 4)` lưu giá trị mỗi cặp .

Target chưa terminal:

, , .

, :

- Target bằng .
- TD error bằng .
- Q mới bằng .

HỎI NHANH

Đề chỉ cấp `env.reset()` và `env.step(a)`, trả về trạng thái mới, phần thưởng và cờ kết thúc. Không có bảng chuyển, cũng không có bản đồ. Cách nào chạy được ngay?

A Value iteration, vì phương trình Bellman đúng cho mọi MDP

B Value iteration, sau khi coi mọi bằng nhau

C Q-learning trên những transition thu được khi gọi `step`

D Dynamic programming theo thứ tự topo của các trạng thái

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q5.1(iv)

Terminal có target riêng

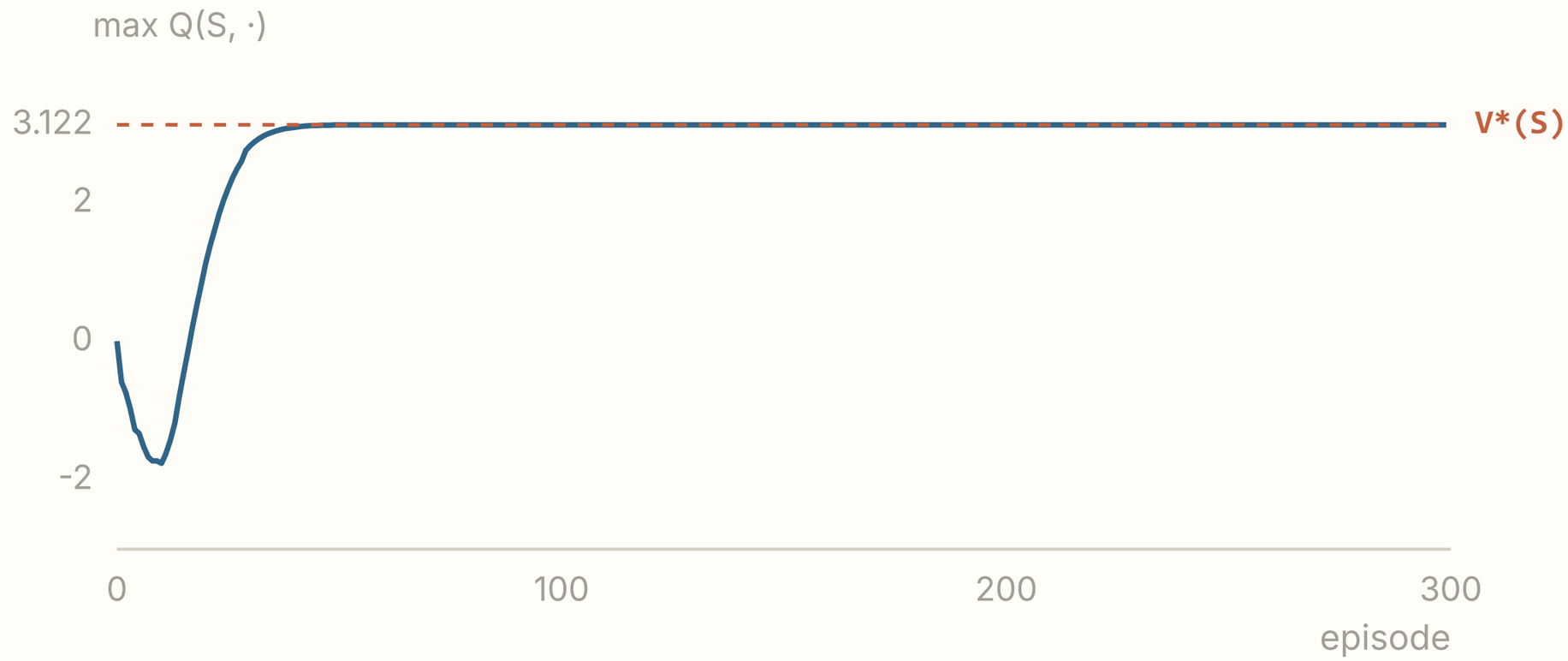
```
def td_target(reward, next_values, terminated):
    if terminated:
        return reward
    return reward + gamma * np.max(next_values)
```

Thưởng terminal 10, target bằng 10.

Giới hạn thu mẫu bên ngoài MDP vẫn cho phép bootstrap.

Trong Gymnasium: đọc riêng `terminated` và `truncated`.

Q tại S tiến tới V*



Q-learning của bài (cùng seed 7, α = 0.3, ε = 0.2): giá trị lớn nhất của Q tại S qua 300 episode đầu tiến tới V*(S) = 3.122 mà value iteration đã tính.

Exploration thu thêm thông tin

```
if rng.random() < epsilon:
    action = int(rng.integers(4))
else:
    best = np.flatnonzero(
        Q[state] == Q[state].max())
    action = int(rng.choice(best))
```

Exploration thử hành động để biết thêm về environment.

Search mở rộng trạng thái bằng quy tắc đã biết.

Chọn ngẫu nhiên mà không cập nhật Q hoặc policy chưa tạo ra phần học.

HỎI NHANH

Epsilon-greedy có 4 hành động, đúng một hành động tốt nhất, , và nhánh explore chọn đều cả 4 hành động. Xác suất chọn hành động tốt nhất là bao nhiêu?

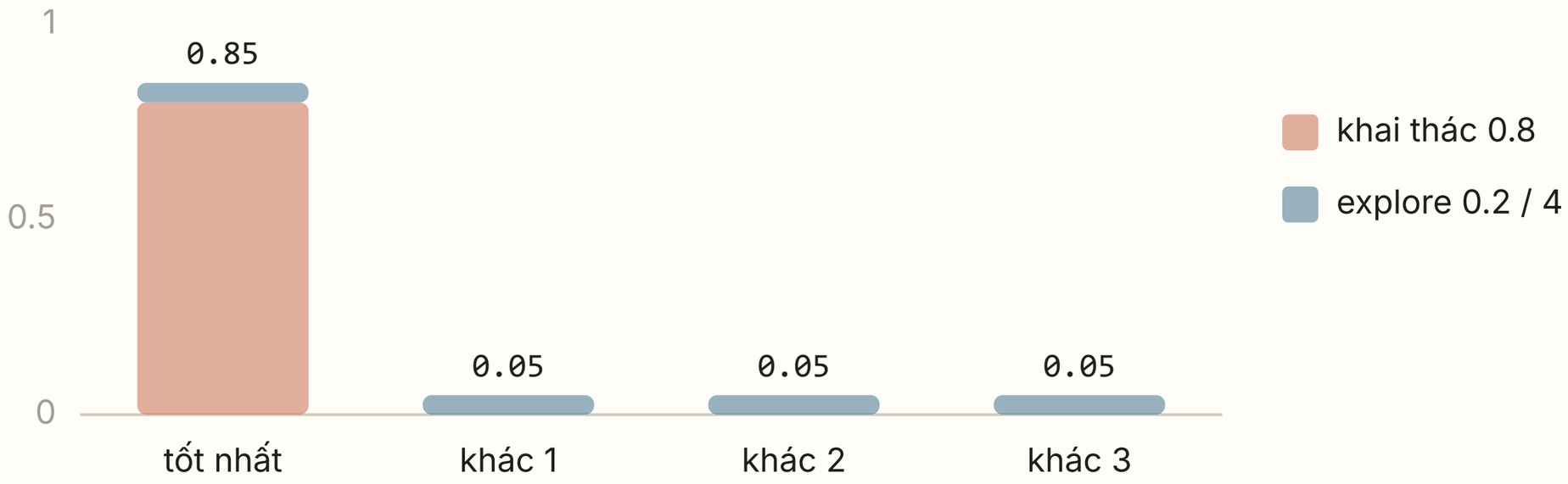
A

B

C

D

Xác suất của epsilon-greedy



Epsilon-greedy với ε = 0.2 và 4 hành động: hành động tốt nhất nhận 0.8 từ nhánh khai thác cộng 0.05 từ nhánh chọn đều, tổng 0.85; mỗi hành động khác 0.05.

Tin tốt lan ngược chậm

Episode đầu, , : chỉ cặp cuối nhận target có thưởng 10.

Các cặp trước đã cập nhật khi phía sau còn bằng 0.

Mỗi lần đi lại cùng đường, thông tin lùi thêm một bước.

Experience replay: lưu transition, cập nhật lại nhiều lần.

HỎI NHANH

Episode đầu tiên đi 5 transition từ S tới đích, và tất cả đều được lưu lại. Q-learning với , khởi tạo bằng 0. Phát lại 5 transition này theo thứ tự nào để tại S đạt chỉ sau một lượt quét?

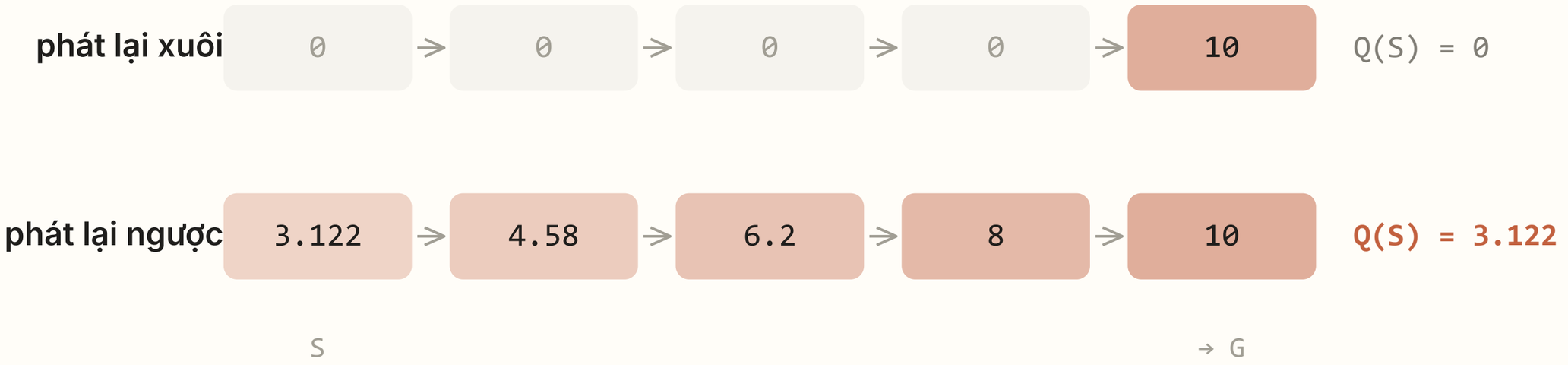
A Đúng thứ tự đã đi, từ S tới đích

B Ngẫu nhiên, để các mẫu bớt tương quan

C Chỉ transition cuối cùng, lặp lại 5 lần

D Ngược từ transition cuối cùng về S

Phát lại xuôi và ngược



Năm transition của episode đầu, α = 1, Q khởi tạo 0: phát lại xuôi thì chỉ cặp cuối nhận thưởng 10; phát lại ngược thì mỗi cặp dùng giá trị vừa tính phía sau, Q tại S lên 3.122 sau một lượt.

Đánh giá policy bằng một đường đi

```
print("Hành động:", " ".join(actions_taken))
print(
    f"Số bước: {len(actions_taken)}, "
    f"return: {discounted:.3f}"
)
```

KẾT QUẢ

```
Hành động: R R U U R
Số bước: 5, return: 3.122
```

Evaluation tắt exploration và không cập nhật Q.

Đường ngắn nhất dài 5 bước: tối ưu từ S.

Kiểm cả policy: chạy từ mọi ô, so với .

HỎI NHANH

Một bảng Q mới học 20 episode. Đánh giá greedy từ S cho đường cho đến hết 30 lượt mà không tới đích. Ta kết luận gì, và nên làm gì?

A Environment sai, vì policy greedy luôn phải tới được đích

B Bật lại khi đánh giá để robot thoát khỏi vòng lặp

C Q chưa học xong ở hai ô này, nên train thêm rồi đánh giá lại

D Return bằng 0 vì không tới đích, nên bỏ hẳn lần đánh giá này

04

Học trực tiếp policy

Policy gradient trên nhiệm vụ một bước

Trái nhận 1 điểm, phải nhận 3 điểm, rồi episode kết thúc.

Bandit này giúp thấy đạo hàm. Nó chưa có vấn đề gán công lao qua nhiều bước.

Tính tay một bước policy gradient

cho và .

Gradient ascent với :

KẾT QUẢ

```
Một bước exact gradient: theta=0.0500, p=0.5125
```

Sau cập nhật, xác suất chọn phải tăng từ lên khoảng .

Muốn tăng thưởng thì dùng dấu cộng.
Muốn giảm loss thì dùng dấu trừ.

HỎI NHANH

Ta muốn cực đại hóa kỳ vọng thưởng , và đã có `gradient` bằng . Dòng cập nhật nào đúng?

A `theta += lr * gradient`

B `theta -= lr * gradient`

C `theta += lr * abs(gradient)`

D `theta = lr * gradient`

REINFORCE ước lượng gradient từ một mẫu

Mẫu gradient:

$$\widehat{\qquad\qquad}$$

Với Bernoulli, .

```
reward = 3.0 if action else 1.0
sample_gradient = (
    (reward - 2.0) * (action - probability)
)
theta_policy += 0.05 * sample_gradient
```

Baseline không phụ thuộc vào hành động được chọn.

HỎI NHANH

Bandit của bài, với nên , và baseline . Mẫu vừa rút chọn **trái** và nhận 1 điểm. Sau một cập nhật REINFORCE như trong code, xác suất chọn phải thay đổi thế nào?

A Giảm, vì mẫu chọn trái được cộng một thưởng dương

B Giữ nguyên, vì 1 điểm vẫn là thưởng dương

C Giảm, vì âm khi chọn trái

D Tăng, vì trái thấp hơn baseline nên bị đẩy xuống

Kết quả sau 400 mẫu

KẾT QUẢ

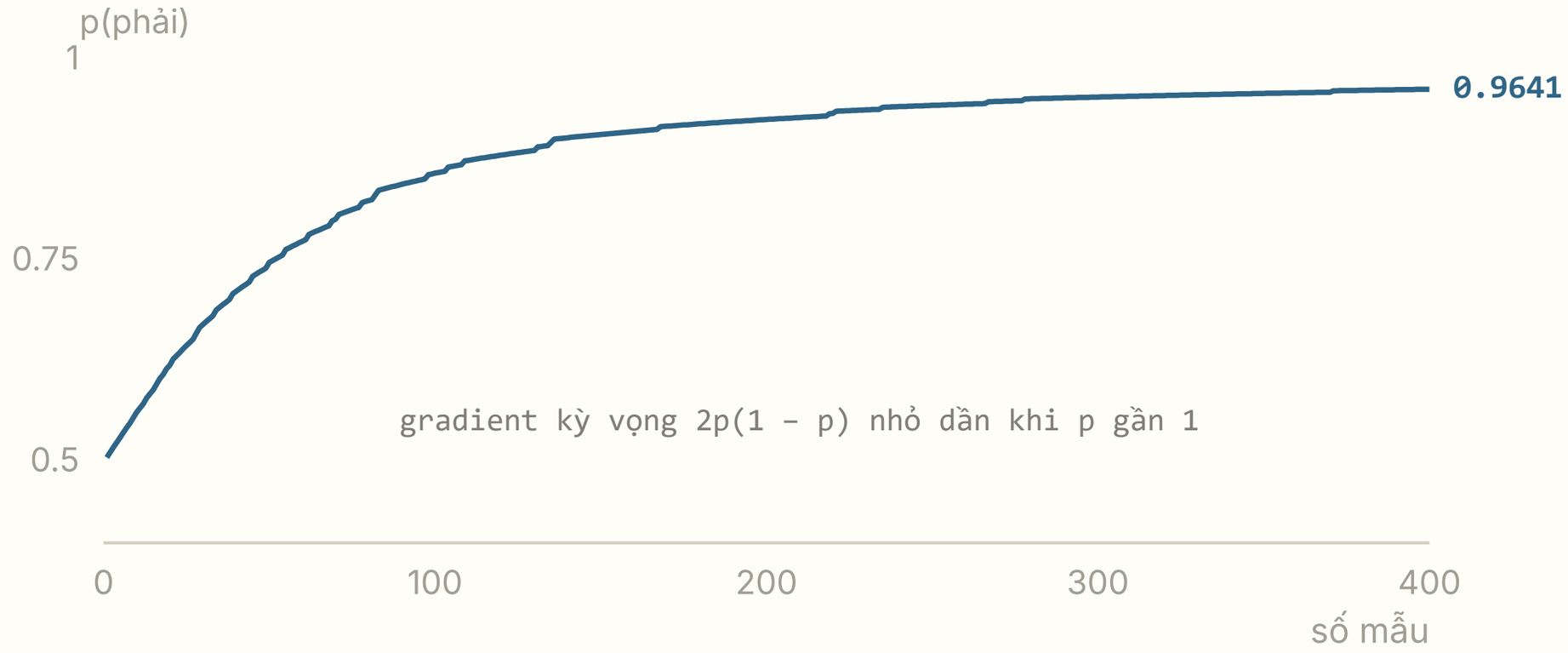
```
Policy sau 400 mẫu: p(phải)=0.9641, E[R]=2.9282
```

Kỳ vọng tính bằng công thức của bandit.

Kỳ vọng chưa bằng mức tối đa 3 vì policy vẫn còn ngẫu nhiên.

Gradient nhỏ dần khi gần 1: càng chắc chắn càng học chậm.

Xác suất chọn phải qua 400 mẫu



REINFORCE với baseline 2 trên bandit của bài (seed 11): xác suất chọn phải tăng nhanh lúc đầu rồi chậm dần, đạt 0.9641 sau 400 mẫu, vì gradient kỳ vọng 2p(1 − p) nhỏ dần khi p gần 1.

Episode nhiều bước: , baseline, critic

^

Hành động ở bước chỉ ảnh hưởng thưởng về sau, nên dùng .

Baseline không đổi kỳ vọng, chỉ bớt dao động; thường là ước lượng

.

Actor-critic: thay bằng .

Kiểm gì trước khi huấn luyện

- Thưởng hoặc terminal sai có thể tạo vòng thu thưởng không đúng mục tiêu.
- Dừng thu mẫu (`truncated`) khác kết thúc thật (`terminated`).
- Reward trong exploration khác kết quả của policy greedy.
- Môi trường ngẫu nhiên: cố định làm Q dao động; đánh giá nhiều episode, nhiều seed.

HỎI NHANH

Đội lưu checkpoint mỗi 100 episode, và chọn checkpoint có tổng thưởng trung bình cao nhất trong 100 episode train gần nhất, với . Vấn đề chính là gì?

A Không có vấn đề, vì đó là reward thật do environment trả về

B Thưởng ấy lẫn cả hành động ngẫu nhiên, nên cần một lần đánh giá greedy riêng

C Nên chọn checkpoint cuối cùng, vì Q-learning luôn tốt dần lên

D Nên tăng lên 1 để đo mọi checkpoint một cách công bằng

Khi đi thi

- Có quy tắc hoặc bảng chuyển: planning (BFS, DP, value iteration).
- Chỉ có `reset` và `step`: học từ tương tác; ghi giới hạn số lần gọi.
- Kiểm `step` bằng tay: va tường, vào đích, hết lượt.
- Dựng oracle trên bản nhỏ để đối chiếu agent.
- Đánh giá greedy riêng; không dùng reward lúc explore.
- Câu tính tay: ghi rõ , , terminal, thứ tự cập nhật.

05

Luyện đề

LUYỆN ĐỀ 1

Sàn trơn. Từ X, lệnh “tiến” đi một ô tới Y với xác suất 0.8, trượt thẳng vào đích với xác suất 0.2. Từ Y thì luôn vào đích. Vào đích nhận 10, bước khác nhận 0, . bằng bao nhiêu?

A

B

C

D

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q3.9

LUYỆN ĐỀ 2

Q khởi tạo 0, , . Nhận lần lượt (A, M, 0, B), (B, M, 0, C), (C, M, 1, A), (A, M, 0, B), (B, M, 0, C). Những ô nào của Q khác 0?

A Chỉ

B , , và

C và

D của A, B, C đều bằng 1

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q7(a)

LUYỆN ĐỀ 3

Bandit của bài, , mẫu REINFORCE là .
So sánh với .

A Cùng kỳ vọng, phương sai là 1 so với 0

B Cùng kỳ vọng, cùng phương sai bằng 1

C Kỳ vọng 1.5 với , 0.5 với

D kéo kỳ vọng về 0, giữ 0.5

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q5.3(a)

Chốt ý

Thưởng, và terminal định nghĩa bài toán; kiểm chúng trước khi train.

Bellman nối hiện tại với tương lai.

Value iteration dùng mô hình; Q-learning dùng transition.

Policy gradient chỉnh trực tiếp xác suất hành động; baseline giảm dao động.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; thêm xác suất trượt vào `step`, đoán trước giảm ra sao rồi kiểm bằng value iteration.